

# Implementation and evaluation of a tool for automated testing of performance and resource con- sumption in Java

---

*Implementering och evaluering av ett verktyg för automatiserad  
testning av prestanda och resurskonsumtion i Java*

**Nils Johansson**

Supervisor : Mina Niknafs  
Examiner : Zebo Peng

External supervisor : Fabian Wiberg, Tobias Hammerslag

## Upphovsrätt

Detta dokument hålls tillgängligt på Internet - eller dess framtida ersättare - under 25 år från publiceringsdatum under förutsättning att inga extraordinära omständigheter uppstår.

Tillgång till dokumentet innebär tillstånd för var och en att läsa, ladda ner, skriva ut enstaka kopior för enskilt bruk och att använda det oförändrat för ickekommersiell forskning och för undervisning. Överföring av upphovsrätten vid en senare tidpunkt kan inte upphäva detta tillstånd. All annan användning av dokumentet kräver upphovsmannens medgivande. För att garantera äktheten, säkerheten och tillgängligheten finns lösningar av teknisk och administrativ art.

Upphovsmannens ideella rätt innefattar rätt att bli nämnd som upphovsman i den omfattning som god sed kräver vid användning av dokumentet på ovan beskrivna sätt samt skydd mot att dokumentet ändras eller presenteras i sådan form eller i sådant sammanhang som är kränkande för upphovsmannens litterära eller konstnärliga anseende eller egenart.

För ytterligare information om Linköping University Electronic Press se förlagets hemsida <http://www.ep.liu.se/>.

## Copyright

The publishers will keep this document online on the Internet - or its possible replacement - for a period of 25 years starting from the date of publication barring exceptional circumstances.

The online availability of the document implies permanent permission for anyone to read, to download, or to print out single copies for his/hers own use and to use it unchanged for non-commercial research and educational purpose. Subsequent transfers of copyright cannot revoke this permission. All other uses of the document are conditional upon the consent of the copyright owner. The publisher has taken technical and administrative measures to assure authenticity, security and accessibility.

According to intellectual property law the author has the right to be mentioned when his/her work is accessed as described above and to be protected against infringement.

For additional information about the Linköping University Electronic Press and its procedures for publication and for assurance of document integrity, please refer to its www home page: <http://www.ep.liu.se/>.

## Abstract

Detecting performance issues in software systems is desirable and sometimes critical. In this thesis, a new tool is presented for the purpose of testing performance at a smaller scale. This tool, JFRNG, is implemented as an extension to the Java testing framework TestNG with the purpose of allowing developers to test the performance of Java code in unit and integration tests. The thesis also contains a study of the current methods and approaches to automating the process of detecting performance issues.

An evaluation of JFRNG is performed both in terms of performance and usability, where the tool is compared to JfrUnit, a similar tool that targets the JUnit framework. A usability survey took place during the usability evaluation where professional Java developers gave their opinions on the clarity of the code that is produced by JFRNG and JfrUnit. The survey participants found the code produced by JFRNG to be easier to read and understand and also rated JFRNG slightly higher on a number of specific aspects related to the clarity of the code. The added performance overhead that was observed for JFRNG during the performance evaluation is on par with JfrUnit and does add some run time for tests, suggesting that developers should be mindful when selecting tests that should utilize JFRNG. The functionality is relatively extensive, allowing developers to collect and test for a large number of metrics related to the tested code. Finally, the thesis is concluded with some suggestions for future work, such as integrating more complex methods for detecting performance regressions based on collected metrics.

# Acknowledgments

First, I would like to thank my supervisor Mina Niknafs and my examiner Zebo Peng for their valuable feedback and guidance during this thesis. I would also like to thank Fabian, Tobias and all the other friendly people at Infor for their help and feedback. Finally, I would like to thank my family for their support during all of my studies.

# Contents

|                                                                                       |             |
|---------------------------------------------------------------------------------------|-------------|
| <b>Abstract</b>                                                                       | <b>iii</b>  |
| <b>Acknowledgments</b>                                                                | <b>iv</b>   |
| <b>Contents</b>                                                                       | <b>v</b>    |
| <b>List of Figures</b>                                                                | <b>vii</b>  |
| <b>List of Tables</b>                                                                 | <b>viii</b> |
| <b>1 Introduction</b>                                                                 | <b>1</b>    |
| 1.1 Motivation . . . . .                                                              | 1           |
| 1.2 Aim . . . . .                                                                     | 2           |
| 1.3 Research questions . . . . .                                                      | 2           |
| 1.4 Delimitations . . . . .                                                           | 2           |
| <b>2 Background</b>                                                                   | <b>3</b>    |
| 2.1 Thesis context . . . . .                                                          | 3           |
| 2.2 Java and the JVM . . . . .                                                        | 3           |
| 2.3 Software testing . . . . .                                                        | 4           |
| 2.4 Related work . . . . .                                                            | 5           |
| <b>3 Theory</b>                                                                       | <b>9</b>    |
| 3.1 Java testing frameworks . . . . .                                                 | 9           |
| 3.2 Tools and frameworks for testing performance at a smaller scale in Java . . . . . | 10          |
| <b>4 Method</b>                                                                       | <b>15</b>   |
| 4.1 Pre-study . . . . .                                                               | 15          |
| 4.2 Design . . . . .                                                                  | 15          |
| 4.3 Implementation . . . . .                                                          | 16          |
| 4.4 Evaluation . . . . .                                                              | 16          |
| <b>5 Implementation</b>                                                               | <b>20</b>   |
| 5.1 Architecture . . . . .                                                            | 20          |
| 5.2 UML Sequence diagram . . . . .                                                    | 21          |
| 5.3 Annotations . . . . .                                                             | 22          |
| 5.4 Controlling the recording . . . . .                                               | 24          |
| 5.5 Accessing the recorded data . . . . .                                             | 24          |
| <b>6 Results and discussion</b>                                                       | <b>28</b>   |
| 6.1 Performance evaluation . . . . .                                                  | 28          |
| 6.2 Usability evaluation . . . . .                                                    | 34          |
| 6.3 Implementation . . . . .                                                          | 37          |
| 6.4 Method . . . . .                                                                  | 37          |

|          |                                       |           |
|----------|---------------------------------------|-----------|
| 6.5      | The work in a wider context . . . . . | 38        |
| <b>7</b> | <b>Conclusion</b>                     | <b>39</b> |
| 7.1      | Research question 1 . . . . .         | 39        |
| 7.2      | Research question 2 . . . . .         | 39        |
| 7.3      | Future work . . . . .                 | 40        |
|          | <b>Bibliography</b>                   | <b>41</b> |

# List of Figures

|     |                                                                                                                                                  |    |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1 | The progression of testing levels illustrated. . . . .                                                                                           | 4  |
| 4.1 | Example survey question in google forms. . . . .                                                                                                 | 18 |
| 5.1 | UML sequence diagram that illustrates the execution of a test written with JFRNG. . . . .                                                        | 22 |
| 6.1 | Average execution time of a test suite of 42 tests for different configurations. Time is given in seconds. . . . .                               | 30 |
| 6.2 | Average execution time of a single long running test that compute digits of pi using different configurations. Time is given in seconds. . . . . | 30 |
| 6.3 | Average execution time of the Java Spring pet clinic test suite for different configurations. Time is given in seconds. . . . .                  | 31 |
| 6.4 | Average execution time of the prime number checking test for different configurations. Time is given in seconds. . . . .                         | 31 |
| 6.5 | Answers to "The code in the test is easy to read and understand". . . . .                                                                        | 34 |
| 6.6 | Answers to "It is clear that the code in the test has the behavior given in the test description". . . . .                                       | 35 |
| 6.7 | Answers to "The usage of the annotations at the top of the method is confusing". . . . .                                                         | 35 |
| 6.8 | Answers to "The usage of the 'JfrEvents'/'JfrController' object confused me". . . . .                                                            | 36 |

# List of Tables

|     |                                                                                                                                               |    |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1 | Metrics used in case study by Foo et al. . . . .                                                                                              | 7  |
| 3.1 | Data representing the <code>jdk.ThreadStart</code> event in JFR. . . . .                                                                      | 11 |
| 5.1 | Available recording profiles and the events they enable in JFRNG. . . . .                                                                     | 23 |
| 5.2 | Filter methods available in JFRNG. . . . .                                                                                                    | 25 |
| 5.3 | Methods to retrieve the aggregated values of recorded event fields. . . . .                                                                   | 26 |
| 5.4 | Methods in the <code>JfrResult</code> class that retrieve results from the recording. . . . .                                                 | 26 |
| 5.5 | Methods in the <code>JfrResult</code> class that return boolean values for more convenient assertions. . . . .                                | 27 |
| 6.1 | Average run times for individual tests with and without recording JFR events for the first test suite consisting of 42 smaller tests. . . . . | 32 |
| 6.2 | Average run times for the Java Spring pet clinic tests with and without recording JFR events. . . . .                                         | 33 |



# Listings

|     |                                                                                                                                                                                                                           |    |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.1 | Example of a test written with TestNG where the test method belongs to two test groups and the test class belongs to another test group. The test method is also declared to take a string argument. . . . .              | 10 |
| 3.2 | A microbenchmark written using JMH. The annotations specify that the benchmark uses the AverageTime benchmark mode, outputs the results in microseconds and performs 2 warmup iterations that last 1 second each. . . . . | 10 |
| 3.3 | Small example of how one can print events with the use of a RecordingStream. . . . .                                                                                                                                      | 12 |
| 3.4 | Simple example of a test written with JfrUnit. . . . .                                                                                                                                                                    | 13 |
| 3.5 | Example of a test written using JUnitPerf. . . . .                                                                                                                                                                        | 14 |
| 4.1 | JFRNG test presented in the survey. . . . .                                                                                                                                                                               | 18 |
| 4.2 | JfrUnit test presented in the survey. . . . .                                                                                                                                                                             | 19 |
| 5.1 | Code that triggers the behaviour in the UML sequence diagram seen in figure 5.1. . . . .                                                                                                                                  | 21 |
| 5.2 | Test method that utilizes the @RecordRemote annotation to connect to the server in listing 5.3 . . . . .                                                                                                                  | 23 |
| 5.3 | Example server that can be connected to via the @RecordRemote annotation. . . . .                                                                                                                                         | 24 |
| 5.4 | Example usage of the JFRNG extension. . . . .                                                                                                                                                                             | 25 |



# **1 Introduction**

In this chapter an introduction to the thesis is presented. First, a motivation for the thesis is presented and the context of the work is introduced. Then the aim of the thesis is specified as well as the research questions that will be investigated. Finally the delimitations for the thesis are described.

## **1.1 Motivation**

Ensuring that a software system is stable and responsive is in many cases critical. To ensure good performance, it is naturally desirable to identify and address potential performance issues. Methods and techniques for testing software performance have seen use for a long time, load testing [21] and stress testing [1] are two examples of approaches that test performance at a system level. Testing performance at a smaller scale, at a unit testing level, is another approach to testing performance. This approach is however often ignored [31]. When performance is only tested at a system level, there is a risk that performance issues are spotted late in development. This can become costly as issues spotted late in development can require large refactoring efforts by developers or even architectural changes.

Functional requirements can be tested early in development via unit and integration tests. And with the use of testing frameworks these tests can be automated and integrated seamlessly into the workflow of a developer. This thesis will investigate the possibility of having automated tests for performance related metrics at a unit and integration test level. These types of tests could allow the programmer to verify assumptions on metrics such as allocated memory or number of threads started by methods. This would make it possible to test some performance aspects early in development, and if the tests are automated, developers can continuously verify that assumptions on certain metrics hold and detect possible performance regressions when assumptions are found to be invalid.

There already exist multiple popular testing frameworks that help developers write unit and integration tests, and TestNG is one such framework that is used with the Java programming language. If developers could utilize the TestNG framework to make assertions on metrics related to performance it could provide some additional utility for developers using the framework.

One approach to collecting performance data in Java is utilizing the JDK flight recorder (JFR) framework. Integrating the use of JFR into testing could be one approach to gathering performance data for tests.

## **1.2 Aim**

The aim of this thesis is to develop an extension to the testing framework TestNG that allows developers to write tests that can collect and make assertions on performance related metrics and events for Java applications at the unit and integration test levels.

## **1.3 Research questions**

The thesis will investigate the following research questions:

1. How can an extension to TestNG that allows developers to test resource consumption and performance using JFR be designed and implemented? How much overhead does such an extension add when running tests?
2. What tools, frameworks and methods exist for conducting automated performance regression testing of software systems in general and Java systems in particular? What are their respective approaches to detecting performance regressions?

## **1.4 Delimitations**

There are many metrics that could be used to evaluate the performance of software. The extension developed in this thesis project will be limited to providing testing opportunities with respect to a limited number of performance related metrics. The extension will be developed to support the testing framework TestNG with Java version 16. In the case of integration tests, the extension will be developed to primarily support integration tests where the modules involved in the test all run in the same JVM. More complex environments will not be supported.



## 2 Background

In this chapter the context of the thesis project is first described. Then technical background information that is relevant for the thesis and the research questions is also presented. Finally in section 2.4, work relevant to answer the research questions and other research that is relevant for the thesis is presented.

### 2.1 Thesis context

This thesis is done in collaboration with the company *Infor* at their office in Linköping. The Infor M3 Technology team in Linköping develops software that is essential for deploying, running, and maintaining their M3 ERP product suite. The codebase is written in Java. The code plays a central part in the runtime of their ERP system and ensuring that the code performs well under heavy load is of high importance. As an effort to make development more efficient they are looking for ways to test the performance of the code as early as possible in the development process. They would therefore like to investigate the possibility of testing the performance of Java code at a smaller scale, for example at the unit test level or in a small integration test environment.

### 2.2 Java and the JVM

Java is an object-oriented, class-based, general-purpose programming language that is currently developed by Oracle Corporation [11]. Java code is compiled to Java bytecode which can be run by a Java virtual machine (JVM) that translates bytecode to the machine instructions of the platform. Java bytecode thus functions as an instruction set for the JVM. By compiling to bytecode first, Java can achieve portability as it may run on any platform that has a JVM that can translate bytecode to the platforms instruction set. The reference implementation of the JVM is HotSpot [35], which is developed as a part of the OpenJDK project, but other JVM implementations also exist.

#### Garbage collection in Java

Automatic garbage collection (GC) is a feature of the Java language that may impact performance. In the Java language, objects that are created will be allocated on the heap. Garbage

collection is performed by the JVM to remove objects that are no longer used from the memory heap. Because Java has GC, the programmer does not need to manually allocate and deallocate memory like one does in other programming languages such as C and C++. The JVM may implement garbage collection in any preferred way that conforms to the JVM specification [20], but here we discuss the garbage collection of the HotSpot JVM.

The HotSpot JVM can utilize any of the multiple different GC algorithms, but generally they follow a common process. The first step in a GC process consists of marking the objects on the heap that are still in use, these objects are marked to be *alive*. An object is deemed to still be alive if any part of the program still has a pointer to the object. Objects that are deemed to not be alive will be removed from the heap. After unused objects have been removed, the heap can be compacted so that the alive objects are allocated contiguously without any unused memory in between them.

The HotSpot JVM, for all GC algorithms except ZGC, divides the heap into two *generation* sections to optimize the GC process [10]: a young generation section and a old generation section. The young generation of the heap stores newly created objects. After an object has been alive for a certain time it is moved to the old generation of the heap. When the young generation becomes full, a *minor collection* is triggered where a GC is performed on the young generation section of the heap. When the old generation becomes full, a *major collection* is triggered. During a major collection, a GC is performed on the whole heap. It is worth noting that a GC pauses the execution of a running program. This means that a GC, especially a major collection, can have a negative impact on the performance.

## 2.3 Software testing

When testing software one can generally separate the testing process into three levels: unit testing, integration testing, and system testing [8]. A fourth level, acceptance testing, can also be added. Typically, the different testing levels are introduced at different stages in development as some testing stages require a more complete system. The progression of the test stages can be seen in figure 2.1.



Figure 2.1: The progression of testing levels illustrated.

### **Unit testing**

A unit test is written to test a smaller isolated code unit. For example, in Java, a unit test can be written to validate the expected behaviour of methods and classes. To test code in isolation it is common to simulate parts of a system that contain functionality that the tested unit depends on via mock objects [23]. Unit tests are often automated and written with use of a testing framework. Examples of testing frameworks for Java are JUnit [17] and TestNG [32].

### **Integration testing**

Integration testing occurs when multiple parts of a system are tested to ensure that they interact with each other as intended. For example, the integration of a front end client and a back end server could be tested to ensure that the communication between them and the interface that they use produce the expected results.

### **System testing**

System testing is performed when a system as a whole is tested and all parts are integrated. Functional requirements on the behaviour of the system can be tested at this level. It is also possible for non functional requirements, such as requirements related to usability or performance to be tested at this level.

### **Acceptance testing**

Acceptance testing tests if the complete system is acceptable from the perspective of the client or user. It can for example test if requirements that the client had specified are fulfilled. Acceptance testing can be seen as the last testing stage in a development timeline.

### **Performance regression testing**

A performance regression occurs when a new version of a system performs worse in some aspect. A code change to a system can result in it having a slower response time or consuming more memory for example. Multiple approaches to detecting performance regressions exists. Performing load testing, where the tested system is subjected to a load, and monitoring the performance of the system is one approach. This approach can require manual efforts from developers or performance engineers, both when conducting the test and when analysing the results. One can also use a profiler to collect performance related data from the runtime of an application. Another approach is the utilization of application performance management tools that allows users to monitor a deployed system and its performance properties. As these mentioned methods for the most part only gather data, manual efforts by developers or performance engineers to analyze the data are required to actually spot performance regressions. Approaches to automated detection of performance regressions will be explored further in sections 2.4 and 3.2.

## **2.4 Related work**

Published work related to the thesis and the research questions is presented in this section.

### **Methods for automatic performance regressions detection**

Multiple attempts to utilize machine learning and statistical analysis for the purpose of detecting performance regressions have been made.

Chen and Shang investigate the type of code changes that lead to performance regressions in software systems [5]. They do this by running benchmarks for a large number of commits in two open-source projects, *RxJava* and *Hadoop*. To detect performance regressions the authors collect the following metrics from their benchmarks: response time, CPU usage, Memory usage, I/O write and I/O read. To detect and quantify performance regressions based on the collected metrics, they utilize statistical analysis techniques. Specifically, they use Student's t-test to detect significant deviations between sets of collected performance metrics.

Foo et al. present an approach to automating performance analysis for software systems [9]. The authors utilize *performance signatures* that capture correlations between measured performance metrics. These performance signatures are obtained using data mining techniques on data sets containing results from previously conducted load tests on a system. By obtaining performance signature they are then able to detect performance regressions by comparing previous performance signatures with performance metrics obtained from new performance tests. They conduct three case studies on three different software systems to verify their approach. One case study involves spotting performance regressions when making changes to the Dell DVD Store application [7]. Metrics are collected from three separate parts of the tested system for the purpose of detecting performance problems. The system consists of a load generator, a Tomcat server and a MySQL server. The metrics the authors collect from these subsystems can be viewed in table 2.1. The metrics used for the other case studies are not fully presented by the authors.

Hewson et al. also utilize machine learning techniques to detect performance regressions [13]. In their work they present *Buto*, a tool that collects performance data from applications that are executed on JVMs. Buto collects data at runtime using MBeans [25]. In the paper the following metrics are listed as properties that may be collected and later used to detect performance regressions:

- Number of classes loaded.
- Heap and non heap memory used.
- Thread count.
- Memory used for buffers, thread pools and eden space.
- Old generation memory used.
- Garbage collection counts.

These metrics are collected during a large number of runs and are then used to build a performance model. New test runs can then be compared to the established performance model to detect deviations that may indicate that a performance regression has occurred.

Another model-based approach for detecting performance regressions is proposed by Shang et al. [29]. The performance of a system can be described by a large number of performance counters that collect data for specific modules and functionality. In their work they seek to detect regressions by modelling performance with performance counters while also trying to minimize the number of performance counters needed to model the systems performance. By dividing performance counters in groups of clusters, where a cluster contains correlated performance counters, they determine what counters are necessary for representing the performance of a system. In their approach they build a regression model for one performance counter in each cluster. They then detect performance regressions by using the prediction error, which is obtained using the model of a cluster and new test results from the performance counters in that cluster. Finally, they show in two case studies that their approach successfully detects performance regressions.

| Load generator               | Tomcat server    | MySQL server                |
|------------------------------|------------------|-----------------------------|
| % Processor Time             | % Processor Time | % Processor Time            |
| # Orders/minute              | # Threads        | # Private Bytes             |
| # Network Bytes Sent/sec     | # Virtual Bytes  | # Bytes written to disk/sec |
| # Network Bytes Received/Sec | # Private Bytes  | # Context Switches/sec      |
|                              |                  | # Page Reads/sec            |
|                              |                  | # Page Writes/sec           |
|                              |                  | % Committed Bytes In Use    |
|                              |                  | # Disk Reads/sec            |
|                              |                  | # Disk Writes/sec           |
|                              |                  | # I/O Reads Bytes/sec       |
|                              |                  | # I/O Writes Bytes/sec      |

Table 2.1: Metrics used in case study by Foo et al.

Nguyen et al. [24] utilize control charts, a statistical control technique that can be used to detect deviations, to analyze performance and detect performance regressions. By constructing control charts based on performance counters observed during load tests, control limits for the expected performance of a systems are created. Then testers can spot deviations that may indicate that a performance regression has occurred by comparing performance data acquired in future load tests with the established control limits.

### Performance assertions

In this section a number of research projects that present tools and approaches to writing performance assertions for parts of a software system are presented.

#### SPL

In their paper, Bulej et al. present a framework that utilizes Stochastic Performance Logic (SPL) to define performance requirements for software [2]. By using SPL one can conduct performance testing at the unit testing level. This is realised with their approach where developers write assumptions on the relative performance between methods and spot performance regressions when these assumptions are detected to be invalid. A developer can for example assume that a method  $M_1$  will have a longer execution time relative to another method  $M_2$  and express this assumption using SPL. Because the performance of the tested methods can be assumed to have a similar relation when the testing is performed on other systems, they achieve an approach to performance unit testing that can be portable between different systems.

#### PSpec

Perl and Weihl introduce PSpec [26], a language that can be used to write assertions for software performance properties. PSpec is not integrated into any programming language, instead it serves as a separate language and tool set for processing data logs produced by a system. Performance events and metrics are defined by the programmer and based on events produced in the logs PSpec utilize. Assertions can then be defined on the metrics using PSpec. The assertions are evaluated with the PSpec checker tool that takes a data log as an input and determines if the assertions are true for the given data. The metrics that PSpec places assertions on are typically captured via *intervals*. Intervals are viewed as all events in a log between a start event and an end event.



## PA

Vetter and Worlery introduce another approach to expressing assertions on software performance [36]. Their prototype, *performance assertions* (PA), allows testers and developers to place assertions on sections of the code. These assertions are placed directly in the source code of the tested program rather than in separate test files. The assertions are made with expressions written in their PA language. With their approach, performance metrics are collected at runtime. The type of data that will be collected depends on the assertions that are placed in the source code using the PA language. The assertions are evaluated using the runtime data, and when an assertion fails multiple actions may be taken. For example, the PA runtime may be configured to log the failed assertion or to invoke a user-defined function when an assertion fails.

## Garbage collection

Performance of Java applications will inevitably be impacted by the garbage collection (GC) process. Multiple studies have been conducted where the performance of different garbage collectors have been compared [12, 4, 19]. This thesis is not interested in comparing different GC algorithms, but ways to measure and evaluate the performance of the GC is of interest as GC is a part of any Java application. Obtaining performance metrics related to the GC could provide some utility as it could help locate potential performance issues.

Lengauer and Mössenböck present a method for automatically tuning the GC parameters to increase the performance of the GC for a given application [19]. When evaluating a GC configuration for an application they use four metrics:

- *Garbage collection time*. The time spent collecting garbage.
- *Run time or throughput* (depending on the benchmark tool used).
- *Garbage collection frequency*. The number of garbage collection cycles executed during a benchmark.
- *Peak heap usage*. The peak amount of memory that is live during the execution of the benchmark.

Grgic et al. [12] compare three different GC algorithms. The authors run six benchmark applications with each GC algorithm and compare the algorithms based on metrics obtained from running the benchmarks. The metrics that are used to evaluate the GC algorithms are the number of garbage collections performed as well as the total time spent executing the GC.

Carpen-Amarie et al. compare the performance of six different Java garbage collectors on a multi-core architecture [4]. To evaluate the performance of the GC algorithms in their study they measure the total execution time, the GC pause time and the total number of GC pauses when executing benchmarks from the DaCapo benchmark suite [6]. They also study client response time latency for read and update operations to a Cassandra DB server. They find that the highest latencies occur when garbage collection takes place on the DB server.



## 3 Theory

A review of the tools and frameworks that are relevant for the thesis is presented in this chapter. The Java testing frameworks that are relevant for the thesis are introduced first. Then a description of the tools and frameworks that can be used to test performance at a smaller scale in Java is given in 3.2.

### 3.1 Java testing frameworks

The most popular testing frameworks, JUnit and TestNG, are introduced in this section.

#### JUnit

JUnit [17] is a testing framework for Java. It can be used to write and run unit tests. Test methods are declared by placing a `@Test` annotation above a method. Other annotations such as `@AfterAll` and `@BeforeEach` can be used to further configure the behaviour of test runs.

#### TestNG

TestNG is another testing framework for Java [32]. It is inspired by JUnit. In addition to regular testing features, it adds features such as the ability to group tests in complex ways and pass parameters to test functions. Java annotations are used to declare and configure tests. To declare a test method the annotation `@Test` is placed above the declaration of the method. An example of a test utilizing parameters and groups in TestNG can be viewed in listing 3.1. The framework can be integrated into the Eclipse or IDEA IDE, making it easy for developers to run tests and view test results during the development process. It is also possible to configure tests and modify TestNG behaviour using *listeners* [33]. Listeners can be seen as interfaces that can be implemented to modify the behaviour of test methods and test suites. For example, by creating a class that implements the *IInvokedMethodListener* interface provided by TestNG that overrides the methods *beforeInvocation* and *afterInvocation*, it is possible to add code that will be executed before and after tests.

```

1 @Test(groups = { "exampleTests" })
2 public class TestClass {
3     @Parameters({ "TEST_STRING" })
4     @Test(groups = { "group1", "group2" })
5     public void exampleTest(String text) {
6         int result = someMethod(text);
7         assertTrue(result > 0);
8     }
9 }

```

Listing 3.1: Example of a test written with TestNG where the test method belongs to two test groups and the test class belongs to another test group. The test method is also declared to take a string argument.

## 3.2 Tools and frameworks for testing performance at a smaller scale in Java

Multiple tools and frameworks have been developed with the purpose of testing performance of Java applications in some way. In this section some of the more prominent tools for testing and measuring performance of Java applications at a smaller scale are presented.

### JMH

Java Microbenchmark Harness (JMH) is a framework for writing and running microbenchmarks targeting Java code or code from other languages that are able to run on a JVM. Benchmarks written using JMH can measure different performance metrics depending on the *benchmark mode* used. The metrics that may be collected when utilizing all of the available benchmark modes are:

- Throughput (times per second the benchmark can be executed).
- Average time to execute the benchmark.
- Time for a single execution without any JVM warmup.

Further statistical data such as which percentile an execution time falls into can also be observed. JMH can also collect additional data by using any of the profilers that are included in the JMH framework. For example, the GCProfiler can be added to a JMH run-configuration to capture data related to the GC. JMH utilizes annotations to construct and configure benchmark methods. An example of how one can write a microbenchmark using JMH can be found in listing 3.2.

```

1 @Benchmark
2 @BenchmarkMode(Mode.AverageTime)
3 @OutputTimeUnit(TimeUnit.MICROSECONDS)
4 @Warmup(iterations = 2, time = 1, timeUnit = TimeUnit.SECONDS)
5 public void benchmarkMethod() {
6     //Code to benchmark
7 }

```

Listing 3.2: A microbenchmark written using JMH. The annotations specify that the benchmark uses the AverageTime benchmark mode, outputs the results in microseconds and performs 2 warmup iterations that last 1 second each.

| jdk.ThreadStart |            |
|-----------------|------------|
| Attribute name  | Type       |
| startTime       | long       |
| eventThread     | Thread     |
| stackTrace      | StackTrace |
| thread          | Thread     |
| parentThread    | Thread     |

Table 3.1: Data representing the `jdk.ThreadStart` event in JFR.

## Caliper

Caliper is a tool developed by Google that allows developers to write microbenchmarks for Java code [3]. It is similar to JMH but offers less functionality and is only focused on time measurements. Documentation for Caliper is generally lacking and it sees relatively low usage among developers [31].

## JFR

JDK flight recorder (JFR) is a framework that is used to collect data from applications running on JVMs. With JFR, data is collected in the form of events where each recorded event contains data fields that describes its properties. There are a large number of predefined events that can be recorded using JFR. Examples of events that can be recorded are: garbage collections, thread starts and socket input and output events. But it is also possible to define new custom events and record them. To give an idea of how events are represented, the data that is included and recorded when the `jdk.ThreadStart` event occurs is presented in table 3.1. Furthermore, it is also possible to record stack traces for events with JFR. By analysing the stack trace of an event one can see what method in the codebase triggered the recorded event.

JFR recordings can be started in a JVM that a Java application runs in via the command line by executing the following command:

```
$ java -XX:StartFlightRecording myApp.myClass
```

After the recording is stopped, the result of the recording is written to a JFR recording file that has the `.jfr` file format. To analyse JFR recording files one can use a tool such as JDK Mission Control (JMC) [15] that presents the recorded data in a graphical interface. One of the big advantages of JFR is that it can collect data from running applications while adding little overhead. One of the reasons that makes this possible is the fact that JFR is built into the JVM and designed to inflict as little overhead as possible. JFR is included as a free feature of OpenJDK since the release of OpenJDK 11, previously it was a commercial feature only included in the Oracle JDK. An alternative to starting recordings from the command line is utilizing the JFR API to handle recordings programmatically [22].

## JFR API

The JFR API can be used to start and control JFR recordings. The API is a part of the Java Development Kit (JDK) and provides the ability to write code that profiles Java applications. There are mainly three different types of recordings that can be started using the API, each of them having a separate class in the API. They are the following:

- `Recording`
- `RecordingStream`
- `RemoteRecordingStream`

## Recording

The Recording class offers the ability to start regular recordings that record JFR events to a file on the disk. These recordings are saved to the disk in the .jfr format when the recording is stopped. The .jfr file can then be processed using the JFR API to access the data. One can also analyse the .jfr file using JMC.

## RecordingStream

The RecordingStream class offers the ability to start a stream that produces JFR events and enables the developer to define actions that will be performed when a new event is produced by the stream. The stream doesn't write to a recording file, instead the developer has to decide how to handle the events that are produced by the stream in code. Events are produced by the stream as they occur in the JVM that the stream runs in. An example of a method that prints jdk.ThreadStart events captured via a stream is presented in listing 3.3.

## RemoteRecordingStream

Another similar class that can be used to record JFR events, RemoteRecordingStream, offers the ability to start JFR recordings on remote JVMs and get produced events over a network connection. To setup the RemoteRecordingStream it is required to pass a connection to a MBean server on a JVM that contains a FlightRecorderMXBean. The platform MBean server built into the JVM contains a FlightRecorderMXBean. So establishing a connection to the platform MBean server and passing it to the RemoteRecordingStream is one way to record JFR events on a remote host. It is worth noting that JFR streams are a relatively new feature (as of 2022) and both the RecordingStream and the RemoteRecordingStream classes were introduced with Java 16.

```
1 public void startRecording(RecordingStream rs) {  
2     rs.enable("jdk.ThreadStart");  
3     rs.onEvent(e -> {  
4         System.out.println(e);  
5     });  
6     rs.startAsync();  
7 }
```

Listing 3.3: Small example of how one can print events with the use of a RecordingStream.

## JfrUnit

JfrUnit is a tool that integrates itself with JUnit to allow developers to write assertions on events that are recorded by JFR [16] during tests. Because it is an extension to JUnit, performance regression tests written with the help of JfrUnit can be integrated in the developers workflow in the same way as functional unit test can be. JfrUnit utilizes the JFR API to record events by starting a JFR RecordingStream along with tests. The results of the recording is then accessed via a *JfrEvents* object that exposes methods that can be used to access the recorded events and to make assertions on the recorded events. Users of the extension configure tests via the use of annotations to specify what events to record and to configure the settings for the JFR recording. A simple example illustrating how one can use the extension to make assertions on data recorded with JFR events can be seen in listing 3.4.

## Quickperf

QuickPerf is a library that enables developers to evaluate performance and to some extent also assert performance properties via the use of annotations in Java [27]. It is used in com-

```

1  @JfrEventTest
2  public class JfrTestClass {
3      public JfrEvents events = new JfrEvents();
4
5      @Test
6      @EnableEvent("jdk.ThreadSleep")
7      public void testThreadSleep() throws Exception {
8          Thread.sleep(2000);
9          events.awaitEvents();
10         assertThat(events).contains(event("jdk.ThreadSleep"));
11     }
12 }

```

Listing 3.4: Simple example of a test written with JfrUnit.

bination with a testing framework to evaluate the performance of code inside test methods. The library is compatible with the JUnit framework and to some extent also TestNG, where a more limited set of annotations are available. Performance assertions are written with the use of annotations that the QuickPerf library offer. For example, By declaring the annotations `@ExpectMaxHeapAllocation` and `@ExpectMaxExecutionTime` along with arguments, a developer can make assertions on memory allocation and execution time for a test method.

### ScalaMeter

ScalaMeter is a framework for performing microbenchmarks and performance regression testing [28]. It targets the JVM for both of these activities and can be used in the Java and the Scala programming languages. It offers a variety of configuration options for benchmarking. Via the use of ScalaMeter *measurers* the framework can be configured to measure a variety of metrics such as memory usage, running time of benchmarks and method invocation counts. Via the use of measurers it is also possible to configure measuring options, one can configure the benchmark to ignore garbage collection time or to add noise to measurements for example.

ScalaMeter is also able to conduct performance regression tests. This is realised by ScalaMeter saving performance results and storing them in a directory. When benchmarks run again at a later time, the performance data is compared to the data from previous runs of the benchmark. If there are large deviations in performance, the test output signals for a performance regression.

### JUnitPerf

JUnitPerf is an extension to the JUnit framework that adds features for testing performance at the unit test level [18]. The only metrics that the framework is concerned with is the measured execution time and other metrics that can be derived from the execution time. The configuration of the test and the expected performance is defined via the use of annotations. Example usage can be seen in listing 3.5 The framework can be configured to write detailed performance data reports based on the executed tests as CSV and HTML reports.

```
1 public class JUnitPerfTestClass {
2     @Rule
3     public JUnitPerfRule testRule = new JUnitPerfRule();
4
5     @Test
6     @JUnitPerfTest(threads = 4, durationMs = 20_000, warmUpMs = 2_000,
7         rampUpPeriodMs = 2_000, maxExecutionsPerSecond = 100)
8     public void exampleTest() {
9         //Code to test
10    }
11 }
```

Listing 3.5: Example of a test written using JUnitPerf.



## 4 Method

In this chapter the method for the thesis project is presented. The method can be separated into four phases: pre-study, design, implementation, and evaluation. Each phase is presented in further detail in their own section in this chapter.

### 4.1 Pre-study

This phase provided the knowledge that was needed to make well informed decisions during the design and implementation phase. Specifically, approaches to collecting performance data in Java programs were researched to guide the design and implementation. The pre-study provided insight into the current tools and frameworks that can be used to test performance at the unit test level in Java. It also provided background information on the Java environment where Java programs run. Furthermore, published research papers and articles that were related to the thesis aim and the research questions were investigated.

Relevant related work was mostly found by using Google Scholar. Searching for related work with Google Scholar was initially done by using search queries such as "performance testing java", "unit testing performance", and "performance regression testing java". When relevant published material was found, the "cited by" feature in Google Scholar and the papers that were cited by the work were used to extend the search to find more relevant published material.

### 4.2 Design

In this phase the important architectural decisions that would impact the development were taken. Decisions on the approach to collecting the performance data were made. Here it was decided that the JFR API would be utilized to collect performance data in the form of JFR events. The JFR API was chosen as it provides the ability to record a wide variety of events that gives insight into performance related metrics while also promising low overhead. Most of the functionality the extension would contain and make available to users was decided and specified during this phase. Decisions on how users would interact with the extension to be able to use these features were also taken. Some inspiration was taken from other similar projects, most notably JfrUnit.



A simple prototype was developed to evaluate the feasibility of the design choices. After the prototype was successfully implemented and the choices were deemed to be viable, the implementation phase began.

### 4.3 Implementation

The implementation started by building upon the prototype created during the design phase. Refactoring efforts were made to make the prototype easier to maintain and to make it easier to add features from a developers perspective. Planned features that were specified during the design phase were implemented. During the development, additional ideas for features and the implementation often came up. These ideas were noted in a list that the developer managed. Ideas that had been noted were later implemented if they still seemed reasonable.

All code was written in the Java programming language. The Eclipse IDE was used to write, debug, and run code during the development. The version control tool Git was used during development to manage the source code of the project. The project was managed using Maven, primarily to make it easier to distribute the extension to other projects.

Unit tests were written during development to make sure that the implemented functionality works as intended. It also served as a means to detect if new code changes broke existing functionality.

Documentation were written during the implementation phase to explain and demonstrate the usage of the extension to others. Javadoc comments were written in the source code for methods and classes, with a focus on methods that users would interact with. A *README* file for the project was written during later stages of the implementation phase to provide a guide on the available features and the general usage. The extension that became the product of the implementation phase was named JFRNG. The name is inspired by JFR and TestNG, as JFRNG adds support for recording JFR events in TestNG tests.

### 4.4 Evaluation

For the evaluation of JFRNG, a performance evaluation and a usability evaluation was conducted. The method that was carried out during these evaluations are described in the following sections.

#### Performance evaluation

For codebases that have large test suites the execution times can add up and result in long waiting times when the tests are executed. Trying to minimize the execution time of tests is thus a valuable goal and for that reason the performance of the implementation in this project was evaluated.

JFRNG and JfrUnit offer similar functionality in terms of recording events and can thus be compared when utilizing similar functionality. A comparison of the execution times that are acquired when running equivalent tests written with JFRNG and JfrUnit was included to compare the implementation to an existing similar tool.

Four test suites were used during the performance evaluation. To observe the overhead under different circumstances and also gain insight into the cause of the overhead, two categories of test suites for the performance evaluation was chosen. Two test suites contained a larger number of tests with shorter run times and the other two test suites contained only one test that had a long run time. An effort was made to include a diverse set of tests that tested different functionality to increase the validity of the results.

The first large test suite contained 42 unit tests which were mostly tests that were taken from an open source repository that contains a variety of Java algorithm implementations [34]. The other large test suite consisted of 37 tests from the Java Spring PetClinic application [30].

The first long running test suite consisted of a test that ran an algorithm that computed the 800000 first digits of pi. The other long running test suite consisted of a single large test that runs an algorithm that checks if 200 large numbers are prime numbers.

The execution time of the test suites were observed for three different setups. The first setup used the JFRNG extension to record JFR events during tests. The second setup used the JfrUnit extension to record JFR events during tests. With the JFRNG and the JfrUnit setups, the tests were modified to record events with the 'profile' configuration, execute the test code and then retrieve the result of the JFR recording. The 'profile' configuration is a predefined JFR configuration that enables a large number of events to be recorded. The tests were not modified to actually make assertions on recorded data, instead the evaluation just focused on the overhead from recording JFR events with these libraries. The third and final configuration did not record any JFR events or use any library to record performance metrics, it simply executed the code contained in the test as it would normally.

Each test suite was executed five times with each setup and an average execution time was calculated for each test suite. The tests ran on a system with the following specifications:

- OS: Windows 11
- CPU: AMD Ryzen 5 1600
- Memory: 16 GB

The execution times for the test suites were acquired by summing the execution times of tests that were reported by the testing framework used. As described earlier in the report, JFRNG is used with TestNG and JfrUnit is used with JUnit. Both TestNG and JUnit report the execution times of tests when a test suite is executed. For both of these frameworks, the added overhead of methods that were executed before or after test methods, such as methods that initialize or tear down JFR recordings, were also included. Both JfrUnit and JFRNG contains such methods. This was done by making sure that the timestamps used to mark the beginning and the end of tests were assigned their values at the correct places in the code.

### Usability evaluation

Usability is an important quality for any framework or library that aims to actually be used by other developers. A developer survey that evaluates the clarity of the code produced from JFRNG was thus constructed and sent out. Each survey participant was first given some background information about the survey, JFR, and the thesis project. Then each participant was presented two unit tests: one test written using JFRNG, and another written using JfrUnit. Both tests were functionally equivalent, i.e. they contained identical assertions that tested the same type of data, but they differed in the sense that they were written with different libraries. The participants were also given a short description of the tests that described the functionality of the code. The tests that were presented to participants can be viewed in listing 4.1 and 4.2. It was not revealed which test was written with JFRNG and which was written using JfrUnit.

The survey contained the following statements that were answered by participants on a five step scale from "strongly disagree" to "strongly agree":

- The code in the test is easy to read and understand.
- It is clear that the code in the test has the behavior given in the test description.
- The usage of the annotations at the top of the method is confusing.
- The usage of the 'JfrEvents' / 'JfrController' object confused me.

The code in the test is easy to read and understand. \*

1 2 3 4 5

Strongly disagree Strongly agree

Figure 4.1: Example survey question in google forms.

```

1 public class TestClass {
2
3     public JfrController controller = new JfrController();
4
5     @RecordJfrEvents({
6         ThreadStart.EVENT,
7         GarbageCollection.EVENT,
8         ThreadSleep.EVENT,
9         FileRead.EVENT
10    })
11    @Test
12    public void TestMethod() {
13        Bar bar = new Bar();
14        bar.foo(); //Some method to test
15
16        JfrResult result = controller.stopRecording();
17        String currentThread = Thread.currentThread().getName();
18        long a = result.getFileIORead();
19        long b = result.getThreadsStarted(currentThread);
20        assertEquals(1024, a);
21        assertEquals(12, b);
22        assertTrue(result.hasFieldWithValue(GarbageCollection.CAUSE, "System.gc()"));
23        assertTrue(result.hasFieldWithValue(ThreadSleep.TIME, 1000));
24    }
25 }

```

Listing 4.1: JFRNG test presented in the survey.

The survey participants were professional Java developers at the Infor Linköping office. All responses were anonymous and no further data was collected. The survey was created using Google Forms and distributed to developers at Infor via a link to the survey. An example of how a question looked in the form can be seen in figure 4.1.

```

1  @JfrEventTest
2  public class TestClass {
3
4      public JfrEvents jfrEvents = new JfrEvents();
5
6      @EnableEvent(GarbageCollection.EVENT_NAME)
7      @EnableEvent(ThreadSleep.EVENT_NAME)
8      @EnableEvent(ThreadStart.EVENT_NAME)
9      @EnableEvent(FileRead.EVENT_NAME)
10     @Test
11     public void TestMethod() {
12         Bar bar = new Bar();
13         bar.foo(); //Some method to test
14
15         jfrEvents.awaitEvents();
16         long a = jfrEvents.filter(JfrEventTypes.FILE_READ)
17             .map(event -> event.getLong(FileRead.BYTES_READ.getName()))
18             .reduce(0L, Long::sum);
19         String currentThread = Thread.currentThread().getName();
20         Predicate<RecordedEvent> startedInThisThread = event -> {
21             return event.getThread(ThreadStart.PARENT_THREAD.getName())
22                 .getJavaName()
23                 .equals(currentThread);
24         };
25         long b = jfrEvents.filter(JfrEventTypes.THREAD_START)
26             .filter(startedInThisThread)
27             .count();
28         assertEquals(1024, a);
29         assertEquals(12, b);
30         assertTrue(jfrEvents.contains(
31             JfrEventTypes.GARBAGE_COLLECTION.withCause("System.gc()")));
32         assertTrue(jfrEvents.contains(
33             JfrEventTypes.THREAD_SLEEP.withTime(Duration.ofMillis(1000))));
34     }
35 }

```

Listing 4.2: JfrUnit test presented in the survey.



## 5 Implementation

In this chapter the result of the implementation, JFRNG, is presented. JFRNG is a library that integrates itself with TestNG. It allows developers to record performance related data in tests written using the TestNG framework. Data is recorded in the form of JFR events. As of Java 16 there is a total of 159 different events that can be recorded by JFR, all of them can also be recorded using JFRNG. It is also possible to define new events and record them with JFRNG. In the following sections of this chapter, the library and its functionality is presented in further detail.

### 5.1 Architecture

The most important architectural components of JFRNG is presented in this section.

#### TestNG listener

A TestNG listener is utilized to modify the behaviour of tests. Specifically, the `IInvokedMethodListener` [14] which offers the ability to invoke functionality before and after a test runs is used in the project. In the project it is accomplished by creating the class `JfrListener` that implements the `IInvokedMethodListener` interface. The `IInvokedMethodListener` interface contains two methods with the following method signatures:

- `public void beforeInvocation(IInvokedMethod method, ITestResult testResult)`
- `public void afterInvocation(IInvokedMethod method, ITestResult testResult)`

JFRNG uses these two methods as entry points to modifying the behaviour of tests. Via the `IInvokedMethod` argument the properties of the invoked test method and even the test class are accessed by utilizing Java reflection methods. Notably, the annotations that are present for the invoked test method are accessed via reflection methods.

The `beforeInvocation` method is responsible for creating a configuration for the recording based on the present annotations. It then starts a recording based on the configuration. The `afterInvocation` methods is responsible for stopping all recordings after a test (if they haven't already been stopped).

The listener is automatically added to TestNG test classes if the extension is added to the project. This was accomplished by declaring the implemented listener as a service in the projects META-INF/services directory. The TestNG framework then utilize the Java ServiceLoader class to find the listener and invoke its methods when tests are executed.

### EventRecorder

JFRNG relies on the JFR API to start, configure and stop the recording of data.. The API was described in more detail previously in section 3.2. In JFRNG the EventRecorder class is responsible for interacting with the JFR API. It primarily contains functionality for starting and stopping recordings. Specifically, it handles regular disk recordings and recordings via the RecordingStream and RemoteRecordingStream classes. These recordings are highly configurable. The configuration for a recording started by the EventRecorder class is defined by the RecordingConfig object that the EventRecorder takes in its constructor. The RecordingConfig object is created based on the annotations that are present above a test method.

### JfrController

To be able to start and stop recordings in tests, a JfrController object needs to be declared in the scope of the test class like this:

```
public JfrController controller = new JfrController();
```

The listener class is responsible for attaching an EventRecorder object to the JfrController and starting the recording. But the developer has to decide when to use the declared JfrController to stop the recording and fetch the recording results.

### JfrResult

The JfrResult class is responsible for exposing the result of a recording. When stopping a recording using the JfrController class, a JfrResult object is returned. Using the methods in the JfrResult class one can access all of the collected data from the recording in multiple ways. Example usage can be seen in figure 5.4.

## 5.2 UML Sequence diagram

To illustrate the interactions between the classes during a test, a UML sequence diagram was created. The UML sequence diagram can be viewed in figure 5.1 and the corresponding code that triggers the behaviour illustrated in the diagram can be viewed in listing 5.1

```

1 public class TestClass {
2     public JfrController controller = new JfrController();
3     @RecordJfrEvents
4     @RecordWithProfile(RecordingProfile.MEMORY)
5     @Test
6     public void TestMethod() {
7         Foo foo = new Foo();
8         int barResponse = foo.bar();
9         JfrResult result = controller.stopRecording();
10        result.getAllocatedMemory();
11    }
12 }
```

Listing 5.1: Code that triggers the behaviour in the UML sequence diagram seen in figure 5.1.

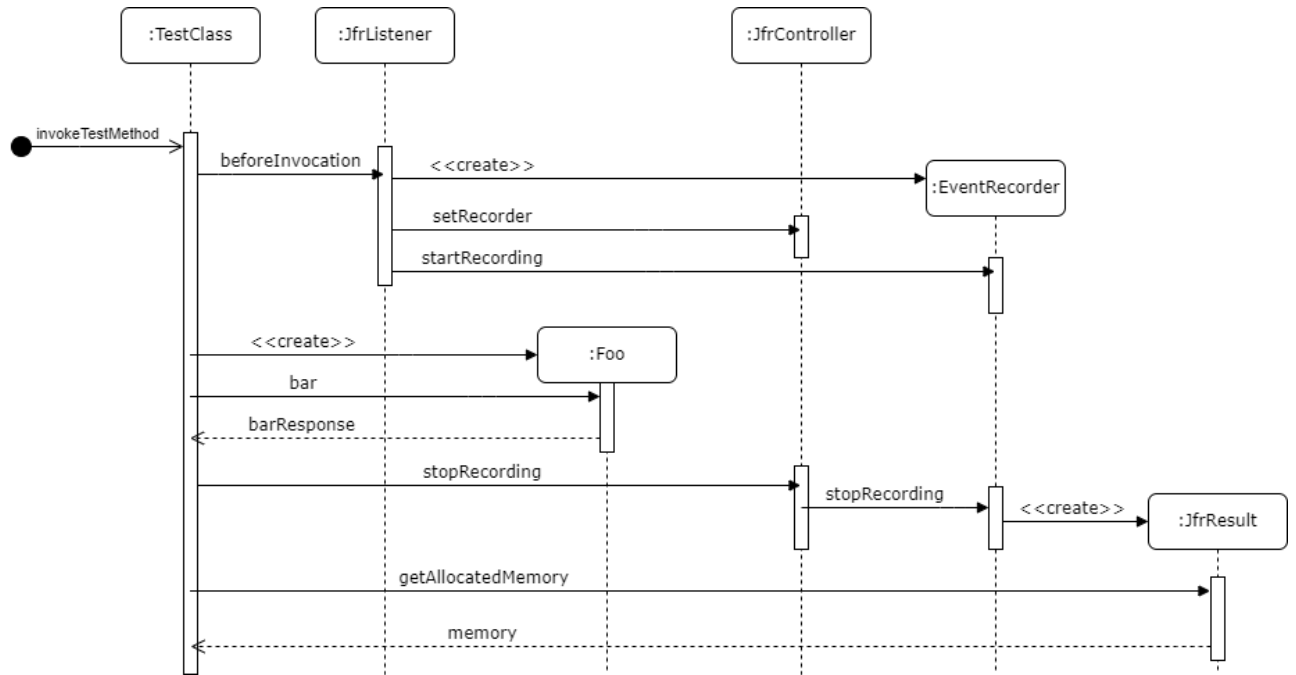


Figure 5.1: UML sequence diagram that illustrates the execution of a test written with JFRNG.

### 5.3 Annotations

JFRNG uses Java annotations to enable JFR recordings for tests and to configure the settings of a recording. Annotations are placed above declared test methods, i.e. TestNG test methods that are declared with the `@Test` annotation. The annotations that can be used to configure test methods in the JFRNG library are presented in the following subsections.

#### @RecordJfrEvents

The `@RecordJfrEvents` annotation enables JFRNG for the test method. As an optional argument it can take an array containing events that should be recorded during the test. This way the developer can control exactly what JFR events should be recorded. For example, to record garbage collection and thread start events one would add this annotation to the test:

```

@RecordJfrEvents({
    GarbageCollection.EVENT,
    ThreadStart.EVENT
})

```

With just this annotation, a recording will be started using a `RecordingStream`, but note that the events that are enabled with this annotation will be enabled for any other type of recording.

#### @RecordWithProfile

The `@RecordWithProfile` annotation is used to enable a `RecordingProfile` for the recording. Enabling a `RecordingProfile` results in enabling a predefined set of events for the recording. The available `RecordingProfiles` that can be enabled are listed in table 5.1.

In addition to the profiles listed in 5.1, it is also possible to start a recording with any of the two predefined JFR configurations "default" or "profile" that are included in the JDK. To

| MEMORY                      | SOCKET_IO   | FILE_IO   |
|-----------------------------|-------------|-----------|
| ObjectAllocationInNewTLAB   | SocketWrite | FileWrite |
| ObjectAllocationOutsideTLAB | SocketRead  | FileRead  |

Table 5.1: Available recording profiles and the events they enable in JFRNG.

```

1 public class RemoteTest
2 {
3     public JfrController controller = new JfrController();
4
5     @RecordJfrEvents(ThreadStart.EVENT)
6     @RecordRemote(Server.URL)
7     @Test
8     public void testRemoteRecording() throws InterruptedException {
9         Thread.sleep(5000);
10        JfrResult result = controller.stopRecording();
11        assertTrue(result.filterOnClass(Server.class)
12                    .hasEvent(ThreadStart.EVENT));
13    }
14 }

```

Listing 5.2: Test method that utilizes the @RecordRemote annotation to connect to the server in listing 5.3

enable a recording profile, pass the profile when using the annotation. For example to enable the memory profile one would add this annotation:

```
@RecordWithProfile(RecordingProfile.MEMORY)
```

Multiple profiles can be enabled by passing an array like this:

```
@RecordWithProfile({
    RecordingProfile.FILE_IO,
    RecordingProfile.MEMORY
})
```

### @DumpJfrToDisk

To start a recording that will be saved to the disk as a .jfr file, the @DumpJfrToDisk can be used. The annotation takes an optional argument that specifies the path to the recording file that will be created. By default the recording file is saved as "recording.jfr". The configuration options that is specified from other annotations will apply to the disk recording as well.

### @RecordRemote

The @RecordRemote annotation can be used to record JFR events on a remote JVM. The annotation requires the developer to pass a JMX service url that leads to a JMXConectorServer as an argument to the annotation. The given url is used to establish a connection to a JMX-ConnectorServer that is attached to a MBean server with a registered FlightRecorderMXBean. Listing 5.3 contains an example of how such a JMXConnectorServer can be created and listing 5.2 contains an example of how one can start a JFR recording on it with the @RecordRemote annotation.



```

1 public class Server {
2     public final static int JMX_PORT = 1919;
3     public final static String JMX_HOST = "localhost";
4     public final static String URL =
5         "service:jmx:rmi:///jndi/rmi://" + JMX_HOST + ":" +
6         JMX_PORT + "/jmxrmi";
7     public final static String BINDING_NAME = "greeter";
8
9     public static void main(String[] args) throws Exception {
10         initJMXConnectorServer();
11         for (;;) {
12             Thread t1 = new Thread();
13             t1.start(); // Start a thread to trigger a JFR event
14             Thread.sleep(500);
15         }
16     }
17
18     private static void initJMXConnectorServer() throws Exception {
19         LocateRegistry.createRegistry(JMX_PORT);
20         MBeanServer beanServer =
21             ManagementFactory.getPlatformMBeanServer();
22         JMXServiceURL jmxUrl = new JMXServiceURL(URL);
23         JMXConnectorServer connectorServer =
24             JMXConnectorServerFactory.
25                 newJMXConnectorServer(jmxUrl, null, beanServer);
26
27         connectorServer.start();
28     }
29 }

```

Listing 5.3: Example server that can be connected to via the @RecordRemote annotation.

### @DisableStacktrace

By declaring the @DisableStacktrace annotation for a test it is possible to disable the recording of stack traces for JFR events. By default, stack traces are recorded for events where there is a stack trace that can be recorded. Some events, such as the *GarbageCollection* event, do not record stack traces because they might not be triggered by program code and thus there is not any stack trace to record. Disabling the recording of stack traces could reduce overhead as it decreases the amount of data for JFR to record.

## 5.4 Controlling the recording

The recording is controlled via a JfrController object that needs to be declared within the scope of the test class. Any JFR recording that is enabled for a test will start automatically right before the test is invoked. To stop the recording and retrieve the results, the method stopRecording() should be used. To erase the currently recorded events from the recording and continue recording events one can use the reset() method.

## 5.5 Accessing the recorded data

The result of the recording is retrieved as a JfrResult object. The JfrResult class offers a large number of methods that enables the developer to access the results. It is possible to retrieve

```

1 public class JfrngTest {
2     public JfrController controller = new JfrController();
3
4     @RecordJfrEvents(GarbageCollection.EVENT)
5     @RecordWithProfile(RecordingProfile.MEMORY)
6     @Test
7     public void TestMethod() {
8         Bar b = new Bar();
9         b.foo(); //Some method that will allocate memory
10        System.gc();
11
12        JfrResult result = controller.stopRecording();
13        String currentThread = Thread.currentThread().getName();
14        long allocatedMB =
15            result.getAllocatedMemoryInThread(currentThread) / 1_000_000;
16
17        assertTrue(allocatedMB < 150);
18        assertTrue(result.hasEvent(GarbageCollection.EVENT));
19        assertTrue(result.hasFieldWithValue(GarbageCollection.CAUSE,
20            "System.gc()"));
21    }
22 }

```

Listing 5.4: Example usage of the JFRNG extension.

| Method                          | Description                                                                                                                          |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| filterOnField(field, predicate) | Filters the result to only contain events that have a field with the given field type that match the given predicate for that field. |
| filterOnEvent(event)            | Filters the result to only contain the recorded events with the given event type.                                                    |
| filterOnClass(class)            | Filters the result to only contain recorded events where the given class is found in the stack trace.                                |
| filterOnMethod(method)          | Filters the result to only contain recorded events where the given method is found in the stack trace.                               |
| filterByThread(event)           | Filters the result to only contain recorded events where the event is omitted by the given thread.                                   |
| filter(predicate)               | Filters the result to only contain events that match the given predicate.                                                            |

Table 5.2: Filter methods available in JFRNG.

the results as a raw stream of JFR events, but the class also offers methods that enables more convenient access to the results. Methods that can be used to filter the results are presented in table 5.2, methods that can be used to get the aggregated values of event fields are presented in table 5.3, methods that make assertions more convenient are presented in table 5.5 and methods that offer functionality for accessing recorded data is presented in table 5.4. Listing 5.4 gives an example on how some of the functionality of the JfrResult class can be used to make assertions on collected data.

| Method                                           | Description                                                                                                                                                                   |
|--------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| getLongAggregate(field, predicate[optional])     | Get the total sum of the recorded long values belonging to the given field. Can take a predicate as an optional argument to only sum the values that match the predicate.     |
| getDoubleAggregate(field, predicate[optional])   | Get the total sum of the recorded double values belonging to the given field. Can take a predicate as an optional argument to only sum the values that match the predicate.   |
| getIntAggregate(field, predicate[optional])      | Get the total sum of the recorded int values belonging to the given field. Can take a predicate as an optional argument to only sum the values that match the predicate.      |
| getDurationAggregate(field, predicate[optional]) | Get the total sum of the recorded duration values belonging to the given field. Can take a predicate as an optional argument to only sum the values that match the predicate. |

Table 5.3: Methods to retrieve the aggregated values of recorded event fields.

| Method                               | Description                                                                                                                                                                                                                                                                         |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| getFileIORead(filePath[optional])    | Get the total number of bytes read from files. An optional argument can be given to get the bytes read from a specific file.                                                                                                                                                        |
| getFileIOWrite(filePath[optional])   | Get the total number of bytes written to files. An optional argument can be given to get the bytes written to a specific file.                                                                                                                                                      |
| getSocketIORead(port[optional])      | Get the total number of bytes read from sockets. An optional argument can be given to get the bytes read from a specific port.                                                                                                                                                      |
| getSocketIOWrite(port[optional])     | Get the total number of bytes written to sockets. An optional argument can be given to get the bytes written to a specific port.                                                                                                                                                    |
| getThreadsStarted(thread[optional])  | Get the total number of threads started. An optional argument can be given to get the total number of threads started by the given thread.                                                                                                                                          |
| getGCPauseSum(timeunit)              | Get the sum of all garbage collection pauses in the given time unit.                                                                                                                                                                                                                |
| getAllocatedMemory(thread[optional]) | Approximates the number of bytes allocated by returning the sum of the number of bytes allocated outside of the thread local allocation buffer (TLAB) and the size of the allocated TLABs. An optional argument can be given to only get the allocations done by a specific thread. |
| getValues(field)                     | Returns a stream containing all recorded values of the given field.                                                                                                                                                                                                                 |
| stream()                             | Returns a stream containing all recorded events.                                                                                                                                                                                                                                    |

Table 5.4: Methods in the JfrResult class that retrieve results from the recording.

| Method                                           | Description                                                                                       |
|--------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <code>anyMatchPredicate(field, predicate)</code> | Returns true if there is a recorded field of the given field type that match the given predicate. |
| <code>allMatchPredicate(field, predicate)</code> | Returns true if all recorded fields of the given field type match the given predicate.            |
| <code>hasFieldWithValue(field, value)</code>     | Returns true if there is a recorded field with the given field type that has the given value.     |
| <code>hasEvent(event)</code>                     | Returns true if the result contains a recorded event of the given event type.                     |

Table 5.5: Methods in the `JfrResult` class that return boolean values for more convenient assertions.

6

## Results and discussion

In this chapter the results of the thesis are presented and discussed. First, the results of the performance evaluation and the usability survey are presented and discussed. Then the method that was used for the thesis is discussed. Finally, a discussion about the thesis that puts the work in a wider context is presented.

### 6.1 Performance evaluation

First, a test suite consisting of 42 tests was run using three different setups. The average execution time for these setups can be seen in figure 6.1. The average execution time for each individual test with each setup that was run can be seen in table 6.1. A test suite consisting of a single long running test that computed digits of pi was also run with three different setups. The average execution time for each respective setup can be seen in figure 6.2. The individual average run times for the tests in the Spring application can be seen in table 6.2 and the average run times for the Spring test suite can be viewed in figure 6.3. Finally, the average run times for the prime checking algorithm test can be seen in figure 6.4.

The bars in all the previously mentioned figures display the average run times when running the test suites with different setups. Each bar is marked with the tool that is utilized in the setup. In the case of the "No recording" bar, no performance data is collected when running the test suite. The tables that display the individual run times for each test in the larger test suites are divided into columns where each column displays the average run time for each test when the tool at the top of the column is utilized. A column that presents the index of each test is also given. The run time is given in milliseconds in the tables.

### Discussion

As can be seen in figure 6.1, JFRNG as well as JfrUnit increased the execution time of the larger test suite consisting of 42 tests dramatically. For this test suite JFRNG had a slightly shorter execution time than JfrUnit. The individual run times for the tests which can be seen in table 6.1 seem to indicate an increased run time of about 2 seconds when using JFRNG or JfrUnit for most of these tests. This is a very large increase for each individual test as they under normal circumstances, where no recording tool is being utilized, run in a couple of

milliseconds. Very similar results was also acquired from the Java Spring test suite, which can be seen in figure 6.3 and in table 6.2.

The test suites consisting of longer running tests both saw a much smaller relative difference between the different run configurations, which can be seen in figure 6.2 and figure 6.4. For both of these suites, JFRNG had a slightly shorter execution time than JfrUnit. Both JFRNG and JfrUnit increased the running time with only about 3 to 4 seconds. The difference in time between the 'no recording' configuration and the JFRNG and JfrUnit configurations is dramatically smaller than what was seen with the larger test suites that contained a larger number of tests.

The increased run time does not seem to grow very fast as the normal execution time of the code in the test grows. On the other hand, increasing the number of tests caused the run time of the test suites to increase dramatically when utilizing JFRNG or JfrUnit. Based on the results, it is plausible that there is a small overhead in actually recording JFR events, as well as a constant overhead cost for starting and stopping recordings with both libraries. In JFRNG there is a synchronization between the event recorder thread that is started by the RecordingStream class and the test thread that runs the test. This probably causes some overhead when starting or stopping a recording. A similar situation could be the case for JfrUnit as well, as both of these tools utilize the RecordingStream class to record events. But further investigation would have to take place to confirm this.

The *profile* JFR configuration that was used in the evaluation when recording JFR events claims to typically have an overhead of 2% while recording, which is fairly inline with the increased run time that was seen in the longer running test. In this evaluation it would not be unexpected for even more overhead as there is not only the overhead from recording JFR events, there is also overhead that might stem from any further functionality in JFRNG and JfrUnit. In figure 6.1 it is apparent that there is more than 2% overhead for the test suite of 42 tests when using JFRNG or JfrUnit as the execution time increased from 1.89 seconds to 88.09 seconds and 94.61 seconds respectively. This could be caused by further overhead that is introduced by the library, such as a cost that occurs when starting or stopping a recording.

The performance evaluation suggests that it is wise to not enable JFRNG for every unit test as this might increase the running time in a major way if there is a significant number of unit tests in the codebase. A good idea would be to prioritize utilization of JFRNG for tests that are suspected to consume a lot of resources or affect performance.

There are large similarities between the run times gotten from the tests that used JFRNG and JfrUnit. This is not surprising since both of these libraries relies on the JFR API to record events in similar ways. The main difference is that the libraries support different testing frameworks, and this does not seem to cause a significant difference in performance.

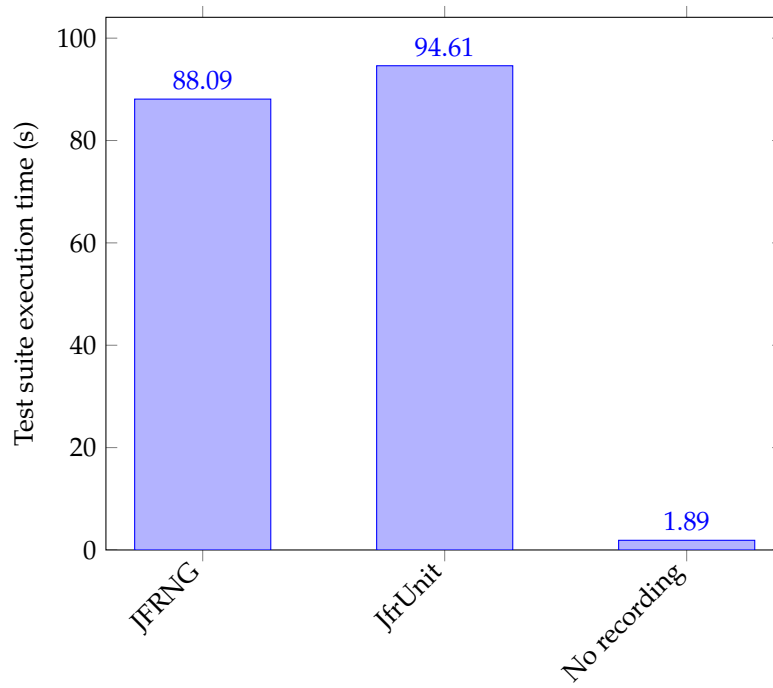


Figure 6.1: Average execution time of a test suite of 42 tests for different configurations. Time is given in seconds.

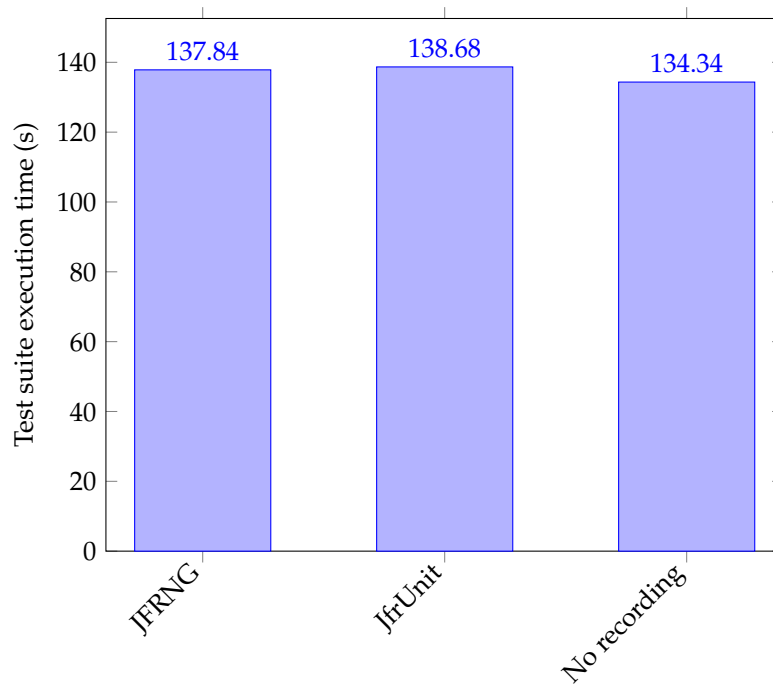


Figure 6.2: Average execution time of a single long running test that compute digits of pi using different configurations. Time is given in seconds.

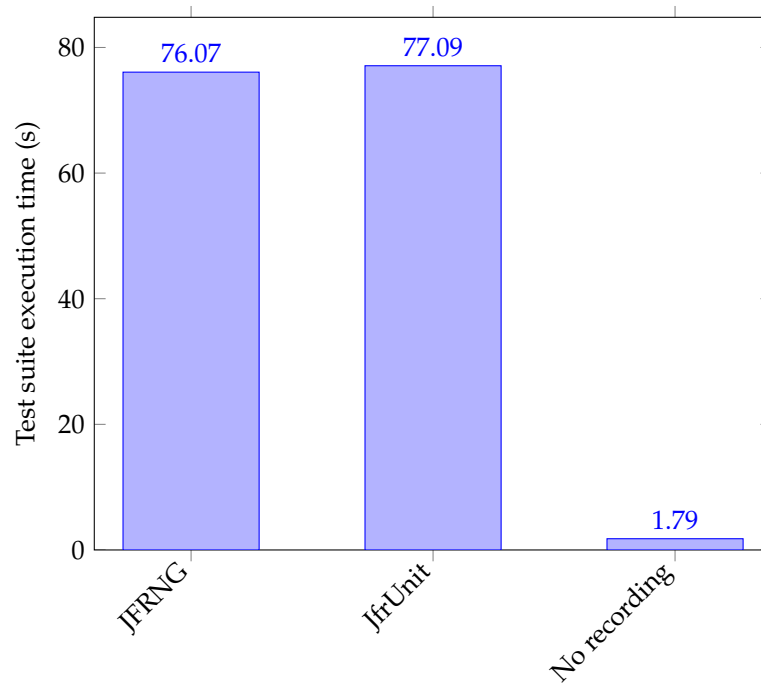


Figure 6.3: Average execution time of the Java Spring pet clinic test suite for different configurations. Time is given in seconds.

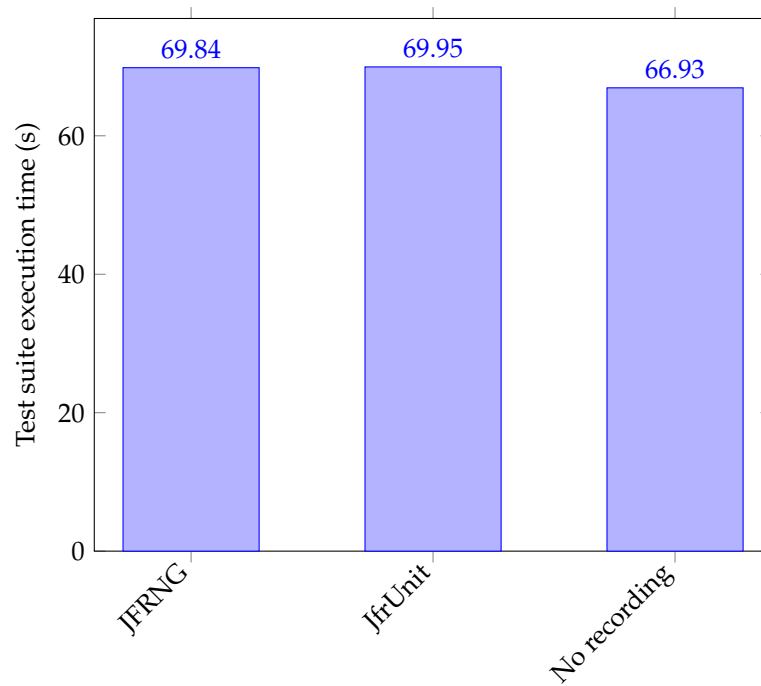


Figure 6.4: Average execution time of the prime number checking test for different configurations. Time is given in seconds.



| Test # | JFRNG     | JfrUnit   | No recording |
|--------|-----------|-----------|--------------|
| 1      | 2057.2 ms | 2271.0 ms | 7.2 ms       |
| 2      | 2067.6 ms | 2242.4 ms | 1.8 ms       |
| 3      | 2060.6 ms | 2266.6 ms | 0.6 ms       |
| 4      | 2065.0 ms | 2277.4 ms | 0.8 ms       |
| 5      | 2066.4 ms | 2225.2 ms | 0.6 ms       |
| 6      | 2067.0 ms | 2243.0 ms | 0.8 ms       |
| 7      | 2070.4 ms | 2217.8 ms | 2.2 ms       |
| 8      | 3078.8 ms | 2720.6 ms | 1584.2 ms    |
| 9      | 2061.6 ms | 2249.2 ms | 0.4 ms       |
| 10     | 2064.2 ms | 2253.0 ms | 0.2 ms       |
| 11     | 2077.8 ms | 2288.2 ms | 226.6 ms     |
| 12     | 2068.6 ms | 2241.0 ms | 0.4 ms       |
| 13     | 2064.4 ms | 2236.4 ms | 0.6 ms       |
| 14     | 2375.8 ms | 2218.4 ms | 3.6 ms       |
| 15     | 2046.0 ms | 2239.2 ms | 1.4 ms       |
| 16     | 2066.4 ms | 2239.6 ms | 0.6 ms       |
| 17     | 2068.2 ms | 2234.4 ms | 1.4 ms       |
| 18     | 2069.0 ms | 2187.6 ms | 1.8 ms       |
| 19     | 2069.4 ms | 2196.8 ms | 0.6 ms       |
| 20     | 2066.2 ms | 2209.0 ms | 1.2 ms       |
| 21     | 2064.8 ms | 2208.2 ms | 0.6 ms       |
| 22     | 2068.2 ms | 2232.0 ms | 2.0 ms       |
| 23     | 2064.8 ms | 2201.2 ms | 2.8 ms       |
| 24     | 2067.6 ms | 2226.6 ms | 1.2 ms       |
| 25     | 2065.0 ms | 2232.4 ms | 1.0 ms       |
| 26     | 2066.8 ms | 2208.0 ms | 0.8 ms       |
| 27     | 2066.8 ms | 2191.6 ms | 2.0 ms       |
| 28     | 2064.0 ms | 2215.8 ms | 0.8 ms       |
| 29     | 2067.6 ms | 2244.8 ms | 0.6 ms       |
| 30     | 2067.0 ms | 2219.4 ms | 0.8 ms       |
| 31     | 2068.6 ms | 2332.4 ms | 8.4 ms       |
| 32     | 2066.2 ms | 2262.4 ms | 0.4 ms       |
| 33     | 2065.2 ms | 2189.0 ms | 4.4 ms       |
| 34     | 2067.6 ms | 2238.2 ms | 10.4 ms      |
| 35     | 2063.8 ms | 2295.8 ms | 1.0 ms       |
| 36     | 2061.4 ms | 2282.0 ms | 1.0 ms       |
| 37     | 2063.4 ms | 2279.4 ms | 1.2 ms       |
| 38     | 2067.4 ms | 2287.6 ms | 0.8 ms       |
| 39     | 2068.2 ms | 2230.8 ms | 0.6 ms       |
| 40     | 2069.6 ms | 2295.2 ms | 1.0 ms       |
| 41     | 2067.2 ms | 2282.2 ms | 0.6 ms       |
| 42     | 2066.2 ms | 2199.6 ms | 1.6 ms       |

Table 6.1: Average run times for individual tests with and without recording JFR events for the first test suite consisting of 42 smaller tests.

| Test # | JFRNG     | JfrUnit   | No recording |
|--------|-----------|-----------|--------------|
| 1      | 2043.6 ms | 2049.4 ms | 10.4 ms      |
| 2      | 2052.8 ms | 2029.6 ms | 54.6 ms      |
| 3      | 2041.6 ms | 2028.4 ms | 10.0 ms      |
| 4      | 2045.2 ms | 2049.6 ms | 16.4 ms      |
| 5      | 2048.4 ms | 2094.2 ms | 26.8 ms      |
| 6      | 2054.2 ms | 2009.0 ms | 15.0 ms      |
| 7      | 2052.8 ms | 2049.0 ms | 35.0 ms      |
| 8      | 2353.6 ms | 2067.8 ms | 8.0 ms       |
| 9      | 2045.6 ms | 2062.4 ms | 1.2 ms       |
| 10     | 2041.6 ms | 2029.4 ms | 41.2 ms      |
| 11     | 2047.8 ms | 2069.2 ms | 60.8 ms      |
| 12     | 2052.8 ms | 2071.4 ms | 14.0 ms      |
| 13     | 2045.8 ms | 1975.2 ms | 123.0 ms     |
| 14     | 2040.4 ms | 2054.2 ms | 23.0 ms      |
| 15     | 2048.6 ms | 2059.2 ms | 20.4 ms      |
| 16     | 2037.0 ms | 2053.2 ms | 33.4 ms      |
| 17     | 2050.6 ms | 2057.4 ms | 24.8 ms      |
| 18     | 2048.8 ms | 2056.8 ms | 54.8 ms      |
| 19     | 2045.0 ms | 2045.0 ms | 24.4 ms      |
| 20     | 2045.2 ms | 2040.8 ms | 1.0 ms       |
| 21     | 2047.0 ms | 2079.6 ms | 23.0 ms      |
| 22     | 2053.4 ms | 2054.4 ms | 20.4 ms      |
| 23     | 2042.4 ms | 2048.0 ms | 10.0 ms      |
| 24     | 2051.2 ms | 2052.8 ms | 15.6 ms      |
| 25     | 2050.0 ms | 2026.8 ms | 8.8 ms       |
| 26     | 2047.6 ms | 2064.6 ms | 25.4 ms      |
| 27     | 2050.8 ms | 2049.2 ms | 33.0 ms      |
| 28     | 2052.4 ms | 2046.4 ms | 68.2 ms      |
| 29     | 2044.4 ms | 2010.8 ms | 7.4 ms       |
| 30     | 2045.6 ms | 2020.8 ms | 22.8 ms      |
| 31     | 2052.8 ms | 2023.4 ms | 8.6 ms       |
| 32     | 2042.4 ms | 2038.0 ms | 146.8 ms     |
| 33     | 2046.8 ms | 2024.6 ms | 8.6 ms       |
| 34     | 2053.4 ms | 2086.4 ms | 74.8 ms      |
| 35     | 2047.6 ms | 2055.0 ms | 60.2 ms      |
| 36     | 2052.6 ms | 2014.0 ms | 83.2 ms      |
| 37     | 2048.4 ms | 3439.8 ms | 572.2 ms     |

Table 6.2: Average run times for the Java Spring pet clinic tests with and without recording JFR events.

## 6.2 Usability evaluation

A usability survey was sent out to 10 developers at Infor. All participants were familiar with Java and unit testing in Java. As mentioned in the method chapter, each participant answered four questions for a test written with JFRNG and another test written with JfrUnit. The answers that the participants gave in the survey can be seen in figures 6.5, 6.6, 6.7 and 6.8. Each bar that can be viewed in these figures represents the number of answers for a given answer alternative. The blue bars represent the answers given for the test where JFRNG is utilized and the red bars represent the answers given for the test where JfrUnit is utilized.

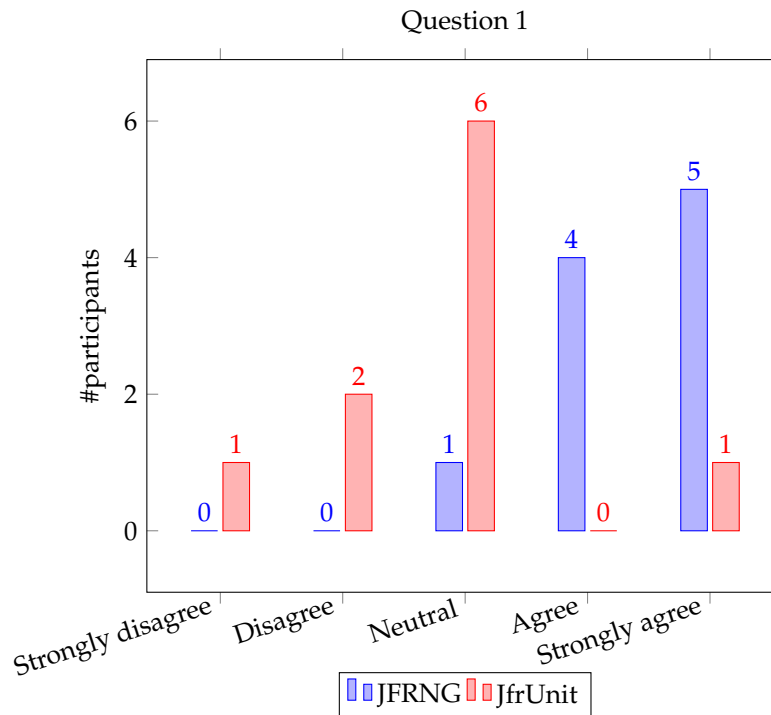


Figure 6.5: Answers to "The code in the test is easy to read and understand".

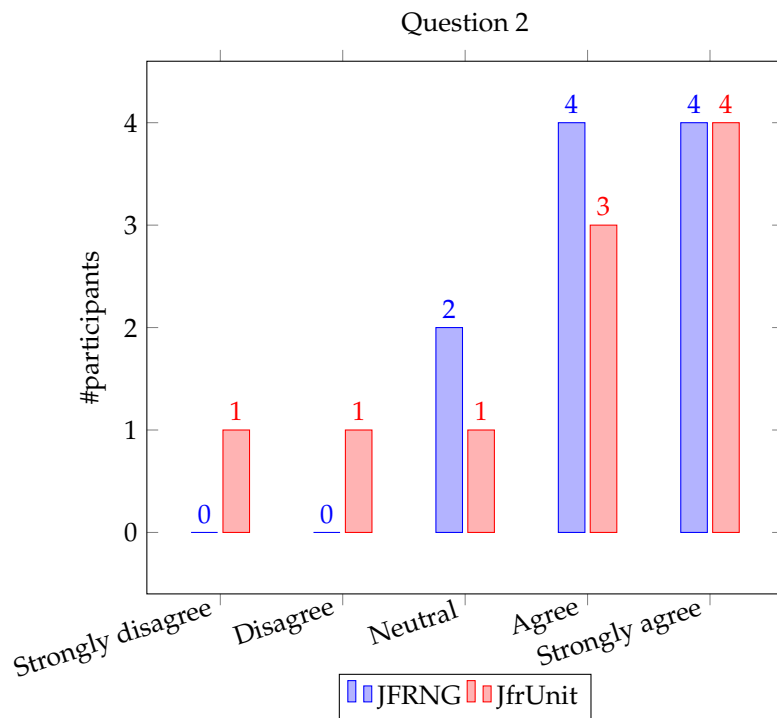


Figure 6.6: Answers to "It is clear that the code in the test has the behavior given in the test description".

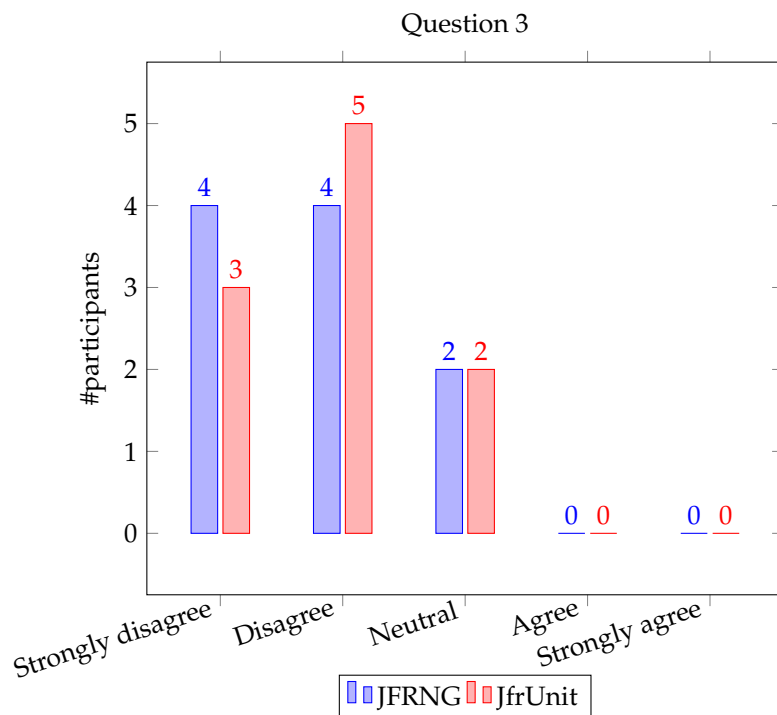


Figure 6.7: Answers to "The usage of the annotations at the top of the method is confusing".

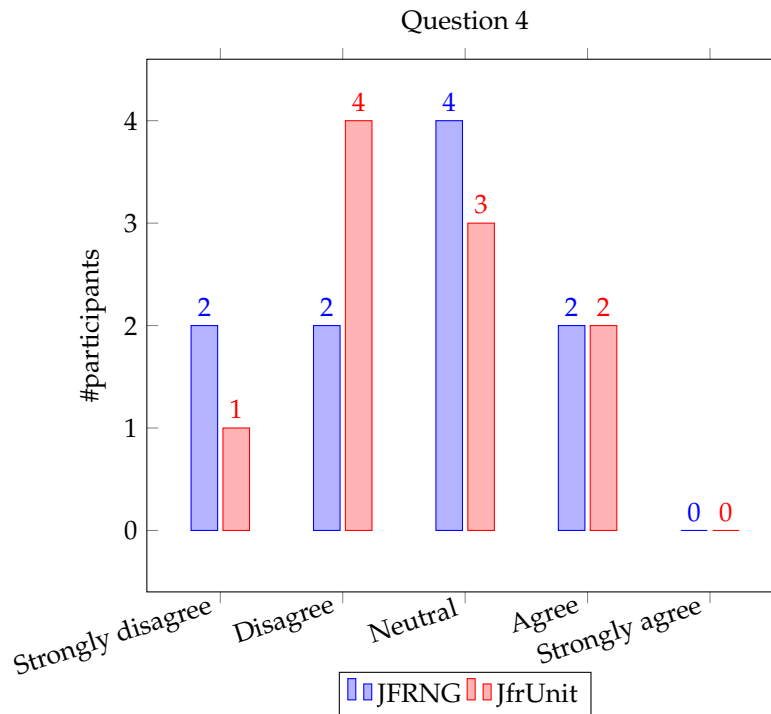


Figure 6.8: Answers to "The usage of the 'JfrEvents'/'JfrController' object confused me".

## Discussion

The usability evaluation was conducted in the form of a survey that was answered by ten Java developers at Infor. The aim of the survey was to gain insight into the usability of JFRNG and compare it to an existing similar library to gain some further perspective.

When the participants were asked if they thought that "The code in the test is easy to read and understand" the responses for the test written with JFRNG were positive and agreed, while the responses for the JfrUnit test were more neutral or disagreeing with the statement, as shown in figure 6.5.

When asked if they agreed with the statement "It is clear that the code in the test has the behavior given in the test description", there were similar responses for both tests. Most participants agreed with the statement for both tests. A few participants disagreed with the statement for the test written with JfrUnit.

Regarding the answers to Question 3, "The usage of the annotations at the top of the method is confusing", The participants didn't think that the annotations at the top of the methods were confusing for any of the tests, and both tools received similar responses.

The last question in the survey investigated whether the usage of the 'JfrEvents' and 'JfrController' objects confused them. Both of these objects have similar functionality and are classes that handles the stopping of JFR recordings in tests. Most answers said that they were not confused by the usage.

Overall, both tools received mostly favourable responses in regards to the clarity of the code produced by them. The responses generally favoured JFRNG a bit more, especially for Question 1 where the participants thought that the code produced by JFRNG was easier to read and understand. The code in the JfrUnit test has more lines of code and is more verbose, so it is not entirely surprising. Although, one could argue that the more verbose code in the JfrUnit examples gives greater insight into what is actually happening.

## 6.3 Implementation

The biggest contribution of this thesis is JFRNG, a tool for collecting and accessing performance data for unit tests. Other design choices could have been made during the implementation and design phase. Another approach to collecting performance data could have been utilized for example. In section 3.2 a number of different frameworks and tools that might have been utilized instead were presented. However, JFR provides a solution that is very powerful and comes with little overhead. It is hard to argue that another solution would have been significantly better.

It is worth to discuss the utility of the functionality that JFRNG offers. It enables developers to record and access performance data in tests. Metrics such as the number of threads started, socket I/O, and bytes allocated can be collected and accessed easily in tests with the help of JFRNG. Creating meaningful tests that actually spot real performance regressions is another challenge. A simple solution could be to establish a baseline for performance metrics and then assert that recorded values in tests for those metrics are within a tolerance interval of the baseline. In section 2.4 a number of more sophisticated approaches for interpreting performance data to detect performance regressions are presented. Using data acquired from running JFRNG in combination with such methods could be worth exploring.

## 6.4 Method

The method used for the work in this thesis has its limitations, which warrants a discussion. A discussion of the method used for the evaluation is presented for both the performance evaluation as well as the usability evaluation. A discussion of the method used during the implementation and the design phase is also given.

### Performance evaluation

There was an attempt to include a variety of tests for the performance evaluation. Because of limited time this was only achieved to a certain extent. A more diverse set of tests or a test suite from a project with a large codebase would have improved the value of the evaluation. An idea was to utilize test suites in the Infor codebase for the evaluation, but most of the codebase is not compatible with Java 16 which is a requirement for JFRNG. So refactoring efforts that were out of scope for this thesis would have been needed for this to work.

The run time of tests were acquired by calculating the difference in milliseconds between the call to `beforeInvocation` and the end of the `afterInvocation` method in the JFR listener. This is a simple but also sufficient method for the purposes of the evaluation. One could run test suites via a microbenchmarking tool such as JMH. This solution would be able to provide a more sophisticated method for measuring the execution time of test suites where optimizations that the JVM makes are ignored.

JFRNG can be configured to run in a wide variety of ways. This performance evaluation only contains one of the basic ways to use the library, i.e. recording a large number of events via a `RecordingStream` and retrieving the results. Further investigations could be made where disk recordings or remote recordings are utilized for example. Configuring JFRNG to use remote recordings would probably lead to increased test run times since it would require communication between at least two JVMs.

### Usability evaluation

The tests that were included in the usability survey display a small portion of the functionality that is available when using the tools. Adding more tests that display more use cases would have provided a more extensive and fairer comparison that evaluates the tools with a larger set of the functionality that is available. The tests used in the survey could have been

written in a number of ways, in programming there are often multiple ways to accomplish the same task. The aim was to write equivalent tests with both tools in the simplest way possible. It is possible that there are other ways to write the test that was written using JfrUnit in other ways that would have been more readable. But this was decided to be the most apparent way to do it from reading the documentation and studying the code base. Writing the simplest test for JFRNG was of course easier since it was developed by the author in this thesis.

Of course, the small number of responses impacts the validity of the survey. The evaluation gives some insight into the usability, but it is hard to draw any conclusions with great confidence when the sample size is this small.

## **6.5 The work in a wider context**

JFRNG helps analyse the performance of a system which is useful for anyone that wishes to spot performance related issues and improve performance. It helps build better software systems in terms of performance which can impact the usability and responsiveness of a system. But the performance of a software system can also have an effect on energy efficiency as more resource consuming systems, for example a system with high CPU load, naturally have a higher energy consumption. Creating systems that are energy efficient is highly desirable, both from an economical and an environmental point of view. Although analysing energy efficiency is not the main purpose of JFRNG, it can still be a contributing factor to achieving it.



## 7 Conclusion

This chapter concludes the thesis. The research questions stated in the introduction are revisited and answered. A discussion about future work based on this thesis is also held.

### 7.1 Research question 1

An extension to TestNG that allows developers to test resource consumption and performance was successfully designed and implemented. It allows developers to test for a wide variety of metrics via JFR events. As stated in chapter 5, the extension uses the JFR API to interact with JFR recordings and a TestNG listener as an entry point to modify the behaviour of tests written with the TestNG framework. JFRNG utilizes Java annotations heavily as a means for users to start and configure JFR recordings for tests. The different ways developers can configure JFR recordings via annotations is presented in detail in chapter 5.

JFRNG does add some amount of overhead for tests. It is particularly noticeable when running smaller tests, but it becomes much more negligible as the run time of the tested code increases. In terms of overhead it is on par with the similar extension JfrUnit, which is seen in the evaluation where there are only minor differences between the tools in added execution times of tests.

### 7.2 Research question 2

In section 2.4 a number of approaches to detecting performance regressions in software systems are presented. A number of research papers propose approaches that involve modelling the performance of a system and using machine learning techniques to detect performance regressions. Other statistical techniques such as the usage of control charts also see some use in this area. Some efforts have been made to develop tools that allow programmers to place performance assertions on systems and thereby detect performance regressions. Approaches presented via related work in this thesis involve placing assertions via the usage of runtime data, first-order logic, and performance logs. Section 3.2 presents a number of Java specific tools and frameworks that enable developers to detect performance regressions at a smaller scale, primarily at the unit testing level. Microbenchmarking tools that primarily measure execution time also provide opportunities for testing performance. JFR and JfrUnit

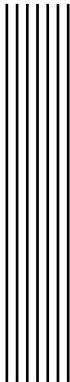


present another approach that focuses more on detecting performance regressions via events recorded in the JVM at runtime.

### 7.3 Future work

In its current state JFRNG allows for some simple methods to automate detection of performance regressions. Extending JFRNG to allow for more sophisticated ways to detect performance regressions from the collected data would be interesting. Collecting data from tests over time and applying machine learning or other statistical techniques such as those mentioned in section 2.4 to detect performance regressions would be one interesting direction for future work.

Expanding JFRNG to contain further support for integration tests would also be a good direction for future work. As of right now JFRNG contains some functionality for recording JFR events on remote JVMs via the `@RecordRemote` annotation. This approach requires a specific connection that utilizes a `JMXConnectorServer` on the remote JVM. It would be useful to support testing in other types of distributed systems too. Expanding JFRNG in such a way would be a good idea. Testing `@RecordRemote` in a real distributed system did not fit in the scope of this thesis, it was only used in a simple setup where all JVMs were on the same machine. An evaluation of the performance and functionality of the remote recording functionality would thus be another good direction for future work.



## Bibliography

- [1] Lionel C. Briand, Yvan Labiche, and Marwa Shousha. “Stress Testing Real-Time Systems with Genetic Algorithms”. In: *Proceedings of the 7th Annual Conference on Genetic and Evolutionary Computation*. GECCO '05. Washington DC, USA: Association for Computing Machinery, 2005, pp. 1021–1028. ISBN: 1595930108.
- [2] Lubomír Bulej, Tomáš Bureš, Vojtěch Hork, Jaroslav Kotrc, Lukáš Marek, Tomáš Trojánek, and Petr Tuma. *SPL: Unit testing performance*. Tech. rep. Citeseer, 2014.
- [3] *Caliper*. URL: <https://github.com/google/caliper>.
- [4] Maria Carpen-Amarie, Patrick Marlier, Pascal Felber, and Gaël Thomas. “A Performance Study of Java Garbage Collectors on Multicore Architectures”. In: *Proceedings of the Sixth International Workshop on Programming Models and Applications for Multicores and Manycores*. PMAM '15. San Francisco, California: Association for Computing Machinery, 2015, pp. 20–29. ISBN: 9781450334044.
- [5] Jinfu Chen and Weiyi Shang. “An Exploratory Study of Performance Regression Introducing Code Changes”. In: *2017 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 2017, pp. 341–352.
- [6] *DaCapo Benchmarks*. URL: <https://dacapo-bench.org/>.
- [7] *Dell DVD Store Database Test Suite*. URL: <https://linux.dell.com/dvdstore/>.
- [8] J. Dooley. *Software Development and Professional Practice*. Apress Series. Apress, 2011. ISBN: 9781430238010.
- [9] King Chun Foo, Zhen Ming Jiang, Bram Adams, Ahmed E. Hassan, Ying Zou, and Parminder Flora. “Mining Performance Regression Testing Repositories for Automated Performance Analysis”. In: *2010 10th International Conference on Quality Software*. 2010, pp. 32–41.
- [10] *Garbage Collector Implementation*. URL: <https://docs.oracle.com/en/java/javase/17/gctuning/garbage-collector-implementation.html>.
- [11] James Gosling, Bill Joy, Guy Steele, Gilad Bracha, Alex Buckley, Daniel Smith, and Gavin Bierman. *The Java language specification*. 2021.
- [12] H. Grgic, B. Mihaljević, and A. Radovan. “Comparison of garbage collectors in Java programming language”. In: *2018 41st International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)*. 2018, pp. 1539–1544.

- [13] Fergus Hewson, Jens Dietrich, and Stephen Marsland. "Performance Regression Testing on the Java Virtual Machine Using Statistical Test Oracles". In: *2015 24th Australasian Software Engineering Conference*. 2015, pp. 18–27.
- [14] *IInvokedMethodListener*. URL: <https://javadoc.jitpack.io/com/github/cbeust/testng/master/javadoc/org/testng/IInvokedMethodListener.html>.
- [15] *JDK Mission Control*. URL: <https://wiki.openjdk.java.net/display/jmc/>.
- [16] *JfrUnit*. URL: <https://github.com/moditect/jfrunit>.
- [17] *JUnit 5*. URL: <https://junit.org/>.
- [18] *JUnitPerf*. URL: <https://github.com/noconnor/JUnitPerf>.
- [19] Philipp Lengauer and Hanspeter Mössenböck. "The Taming of the Shrew: Increasing Performance by Automatic Parameter Tuning for Java Garbage Collectors". In: *Proceedings of the 5th ACM/SPEC International Conference on Performance Engineering*. ICPE '14. Dublin, Ireland: Association for Computing Machinery, 2014, pp. 111–122. ISBN: 9781450327336.
- [20] Tim Lindholm, Frank Yellin, Bracha Gilad, Alex Buckley, and Smith. *The Java® Virtual Machine Specification*. 2021.
- [21] D.A. Menasce. "Load testing of Web sites". In: *IEEE Internet Computing* 6.4 (2002), pp. 70–74.
- [22] *Module jdk.jfr*. URL: <https://docs.oracle.com/en/java/javase/11/docs/api/jdk.jfr/module-summary.html>.
- [23] Shaikh Mostafa and Xiaoyin Wang. "An Empirical Study on the Usage of Mocking Frameworks in Software Testing". In: *2014 14th International Conference on Quality Software*. 2014, pp. 127–132.
- [24] Thanh H.D. Nguyen, Bram Adams, Zhen Ming Jiang, Ahmed E. Hassan, Mohamed Nasser, and Parminder Flora. "Automated Detection of Performance Regressions Using Statistical Process Control Techniques". In: *Proceedings of the 3rd ACM/SPEC International Conference on Performance Engineering*. ICPE '12. Boston, Massachusetts, USA: Association for Computing Machinery, 2012, pp. 299–310. ISBN: 9781450312028.
- [25] *Overview of the Java Dynamic Management Kit*. URL: <https://docs.oracle.com/cd/E19698-01/816-7607/6mdjpi6v1/index.html>.
- [26] Sharon E. Perl and William E. Weihl. "Performance Assertion Checking". In: *Proceedings of the Fourteenth ACM Symposium on Operating Systems Principles*. SOSP '93. Asheville, North Carolina, USA: Association for Computing Machinery, 1993, pp. 134–145. ISBN: 0897916328.
- [27] *QuickPerf*. URL: <https://github.com/quick-perf/quickperf>.
- [28] *ScalaMeter*. URL: <https://scalometer.github.io/>.
- [29] Weiyi Shang, Ahmed E. Hassan, Mohamed Nasser, and Parminder Flora. "Automated Detection of Performance Regressions Using Regression Models on Clustered Performance Counters". In: *Proceedings of the 6th ACM/SPEC International Conference on Performance Engineering*. ICPE '15. Austin, Texas, USA: Association for Computing Machinery, 2015, pp. 15–26. ISBN: 9781450332484.
- [30] *Spring PetClinic Sample Application*. URL: <https://github.com/spring-projects/spring-petclinic>.
- [31] Petr Stefan, Vojtech Horky, Lubomir Bulej, and Petr Tuma. "Unit Testing Performance in Java Projects: Are We There Yet?" In: *Proceedings of the 8th ACM/SPEC on International Conference on Performance Engineering*. ICPE '17. L'Aquila, Italy: Association for Computing Machinery, 2017, pp. 401–412. ISBN: 9781450344043.

- [32] *TestNG*. URL: <https://testng.org/doc/index.html>.
- [33] *TestNG Listeners*. URL: <https://testng.org/doc/documentation-main.html#testng-listeners>.
- [34] *The Algorithms - Java*. URL: <https://github.com/TheAlgorithms/Java>.
- [35] *The HotSpot Group*. URL: <https://openjdk.java.net/groups/hotspot/>.
- [36] J.S. Vetter and P.H. Worley. "Asserting Performance Expectations". In: *SC '02: Proceedings of the 2002 ACM/IEEE Conference on Supercomputing*. 2002, pp. 33–33.